

Báo cáo lỗi — HW06 · 23127195

SUT: EShop ([ttbhanh/eshop-sut](#)) · **Backend:** [http://localhost:3000](#)
Ngày kiểm thử: 2026-09-01 · **Công cụ:** Postman 12.26.1 + Newman 6.2.2 + [curl](#)

Tái hiện độc lập: `bash bugs/reproduce_bugs.sh` — không cần Postman.
Kết quả thô đã lưu tại [evidence/reproduce_output.txt](#) .
Mọi request trong quá trình kiểm thử đều mang header `X-Student-Id: 23127195` .

Tổng quan

24 lỗi trên 3 API, tương ứng **52/144 test case FAIL**.

Mức độ	Số lượng	Mã lỗi
 Critical	4	BUG-A1-01, BUG-A2-01, BUG-A2-02, BUG-A3-01
 High	7	BUG-A1-02, BUG-A2-03, BUG-A2-04, BUG-A2-05, BUG-A3-02, BUG-A3-03, BUG-A3-09
 Medium	8	BUG-A1-03, BUG-A1-04, BUG-A1-05, BUG-A2-08, BUG-A3-04, BUG-A3-05, BUG-A3-08, BUG-A3-11
 Low	5	BUG-A2-06, BUG-A2-07, BUG-A3-06, BUG-A3-07, BUG-A3-10

#	Mã	Mức	API	Vi phạm	Tiêu đề	Issue
1	BUG-A1-01	Critical	FR-04	SEC-06	Leo quyền lên admin qua trường <code>role</code> trong <code>PUT /api/users/me</code>	#5
2	BUG-A1-02	High	FR-04	SEC-01	<code>GET /api/users/me</code> trả về mật khẩu plaintext và <code>reset_token</code>	#6
3	BUG-A1-03	Medium	FR-04	FR-04	Không kiểm tra định dạng số điện thoại	#7
4	BUG-A1-04	Medium	FR-04	FR-04	Không kiểm tra họ tên (rỗng / khoảng trắng / sai kiểu)	#8
5	BUG-A1-05	Medium	FR-04	FR-04	Cập nhật một phần xoá trắng các trường không gửi	#9
6	BUG-A2-01	Critical	FR-09	C4, SEC-02	<code>POST /api/apply-coupon</code> không yêu cầu đăng nhập	#10
7	BUG-A2-02	Critical	FR-09	FR-09	Công thức giảm giá <code>percent</code> sai → giảm giá âm	#11
8	BUG-A2-03	High	FR-09	C3	Ngưỡng đơn hàng dùng <code>></code> thay vì <code>>=</code>	#12
9	BUG-A2-04	High	FR-09	C5	Bỏ <code>user_id</code> là vô hiệu hoá hoàn toàn giới hạn số lượt	#13
10	BUG-A2-05	High	FR-09	C5, SEC-02	IDOR: mượn lượt dùng mã của người khác qua <code>user_id</code>	#14
11	BUG-A2-06	Low	FR-09	FR-09	Thứ tự kiểm tra điều kiện gây thông báo lỗi sai lệch	#15
12	BUG-A2-07	Low	FR-09	—	Mã giảm giá phân biệt chữ hoa/thường	#16
13	BUG-A2-08	Medium	FR-09	—	Tràn số với <code>total_amount</code> lớn	#17
14	BUG-A3-01	Critical	FR-16	SEC-03, FR-12	Người dùng thường import được sản phẩm lên cửa hàng	#18
15	BUG-A3-02	High	FR-16	FR-16	Import không nguyên tử — không rollback khi có dòng lỗi	#19
16	BUG-A3-03	High	FR-16	FR-16	Không kiểm tra <code>price</code> (0 / âm / thiếu / null)	#20
17	BUG-A3-04	Medium	FR-16	FR-16	<code>price</code> là chuỗi phi số vẫn được ghi vào cột số	#21
18	BUG-A3-05	Medium	FR-16	FR-15	Không kiểm tra khoá ngoại <code>category_id</code>	#22
19	BUG-A3-06	Low	FR-16	FR-16	Tên toàn khoảng trắng lọt qua phép kiểm tra	#23
20	BUG-A3-07	Low	FR-16	FR-15	Không giới hạn 255 ký tự cho tên sản phẩm	#24

#	Mã	Mức	API	Vi phạm	Tiêu đề	Issue
21	BUG-A3-08	Medium	FR-16	FR-16	Thiếu <code>category_id</code> bị âm thầm gán mặc định	#25
22	BUG-A3-09	High	FR-16	—	Phần tử <code>null</code> trong mảng gây crash 500 và lộ stack trace	#26
23	BUG-A3-10	Low	FR-16	—	Mất chính xác với <code>price</code> vượt khoảng số nguyên an toàn	#27
24	BUG-A3-11	Medium	FR-16	SEC-04	<code>imageUrl</code> chấp nhận giao thức <code>javascript:</code>	#28

API-1 — FR-04 · Quản lý hồ sơ cá nhân

BUG-A1-01 — Leo quyền lên admin qua trường `role` trong `PUT /api/users/me`

Mức độ	● Critical
Vi phạm	SEC-06 — "API cập nhật hồ sơ không được cho phép thay đổi trường <code>role</code> từ client"; FR-04 — "không thể tự thay đổi thuộc tính <code>role</code> "
Endpoint	<code>PUT /api/users/me</code>
Test case	TC-A1-037 , TC-A1-038

Mô tả. Endpoint cập nhật hồ sơ đọc trường `role` trực tiếp từ body do client gửi lên và ghi thẳng vào cột `role` của bảng `users` . Bất kỳ người dùng đã đăng nhập nào cũng có thể tự nâng mình lên quyền quản trị.

Các bước tái hiện.

```
TOKEN=$(curl -s -X POST localhost:3000/api/login -H 'Content-Type: application/json' \
-d '{"email":"test@eshop.com","password":"Test1234!"}' | jq -r .token)

curl -X PUT localhost:3000/api/users/me \
-H "Authorization: Bearer $TOKEN" -H 'Content-Type: application/json' -H 'X-Student-Id: 23127195' \
-d '{"name":"Test User","shipping_address":"1 Le Loi","phone":"0912345678","role":"admin"}'
```

Kết quả kỳ vọng. Trường `role` bị bỏ qua (hoặc trả `400`); vai trò tài khoản vẫn là `user` .

Kết quả thực tế.

```
{"message": "Profile updated"}
```

```
$ GET /api/users/me      -> role = admin
$ đăng nhập lại         -> JWT mới mang role = admin
$ GET /api/admin/users   -> HTTP 200, trả về toàn bộ danh sách người dùng
```

Tác động. Chiếm quyền quản trị toàn hệ thống chỉ bằng một request. Sau khi leo quyền, tài khoản mở được toàn bộ `/api/admin/*`: xem/xoá người dùng, sửa đơn hàng, sửa giá sản phẩm, quản lý mã giảm giá. Đây là lỗi nghiêm trọng nhất trong toàn bộ đợt kiểm thử.

Vị trí trong mã nguồn. `backend/server.js:114-127`

```
const { name, shipping_address, phone, role } = req.body;    // <- role lấy từ client
...
if (role) { query += ", role = ?"; params.push(role); }      // <- ghi thẳng vào DB
```

Đề xuất sửa. Bỏ hoàn toàn `role` khỏi phép destructuring body. Việc thay đổi vai trò phải là một endpoint admin riêng, có kiểm tra `req.user.role === 'admin'`.

BUG-A1-02 — `GET /api/users/me` trả về mật khẩu plaintext và `reset_token`

Mức độ	● High
Vi phạm	SEC-01 — "Mật khẩu không được lưu dưới dạng plaintext"
Endpoint	<code>GET /api/users/me</code>
Test case	TC-A1-039 , TC-A1-040 , TC-A1-042

Mô tả. Endpoint dùng `SELECT *` nên trả về nguyên vẹn bản ghi người dùng, bao gồm `password`, `reset_token`, `login_attempts`, `locked_until`.

Kết quả thực tế.

```
{"id":2,"name":"Test User","email":"test@eshop.com","password":"Test1234!","role":"user",
  "login_attempts":0,"locked_until":null,"reset_token":null,"shipping_address":null,"phone":null}
```

Tác động. Hai vấn đề chồng lên nhau: (1) mật khẩu trả về **đúng nguyên văn** chứng tỏ hệ thống lưu plaintext, không băm — vi phạm trực tiếp SEC-01; (2) `reset_token` bị lộ là một đường chiếm đoạt tài khoản **độc lập với mật khẩu**. Bất kỳ lỗ hổng XSS hay proxy ghi log nào cũng thu được credential dùng được ngay.

Vị trí trong mã nguồn. `backend/server.js:108-112` (`SELECT * FROM users WHERE id = ?`) và `server.js:22` (`INSERT ... password` không băm).

Đề xuất sửa. Băm mật khẩu bằng `bcrypt`; liệt kê cột tường minh trong câu `SELECT (id, name, email, role, shipping_address, phone)`.

BUG-A1-03 — Không kiểm tra định dạng số điện thoại

Mức độ	● Medium
Vi phạm	FR-04 — "Số điện thoại hợp lệ: bắt đầu bằng số 0 , từ 10–11 chữ số"
Endpoint	PUT /api/users/me
Test case	TC-A1-011 ... TC-A1-020 (10 case)

Kết quả thực tế. Mọi giá trị đều được chấp nhận với {"message":"Profile updated"} :

Giá trị gửi lên	Vi phạm điều kiện	Phản hồi
"abc"	không phải chữ số	200 OK
"12345"	5 chữ số (< 10)	200 OK
"9912345678"	không bắt đầu bằng 0	200 OK
" "	rỗng	200 OK
"0912-345-678"	có ký tự phân cách	200 OK
" +84912345678"	định dạng quốc tế	200 OK
"091234567890"	12 chữ số (> 11)	200 OK
null	null	200 OK

Tác động. Số điện thoại là dữ liệu liên hệ giao hàng. Dữ liệu rác được ghi vào hệ thống mà không có cảnh báo, dẫn tới đơn hàng không liên hệ được với khách.

Đề xuất sửa. Kiểm tra `/^0\d{9,10}$/` trước khi ghi.

BUG-A1-04 — Không kiểm tra họ tên

Mức độ	● Medium · Vi phạm FR-04 · Test case TC-A1-002 , -003 , -004 , -006
--------	---

`name` nhận `"", " ", null` và cả số nguyên `23127195` — tất cả đều trả `200 OK` . Hồ sơ có thể tồn tại với họ tên rỗng, khiến các màn hình hiển thị "Xin chào, " bị trống.

Đề xuất sửa. Bắt buộc `typeof name === 'string' && name.trim().length > 0` .

BUG-A1-05 — Cập nhật một phần xóa trắng các trường không gửi

Mức độ	● Medium · Endpoint PUT /api/users/me · Test case TC-A1-028
--------	---

Mô tả. Câu `UPDATE` luôn gán cả ba cột `name` , `shipping_address` , `phone` . Nếu client chỉ gửi một trường, hai trường còn lại nhận `undefined` và bị ghi thành `NULL` .

Kết quả thực tế.

```
trước:           name='Baseline'      phone='0912345678'  addr='227 Nguyen Van Cu'
PUT {"name":"Chi doi ten"}
sau:             name='Chi doi ten'    phone=None          addr=None
```

Tác động. Mất dữ liệu âm thầm. Một form "đổi tên hiển thị" ở client sẽ xoá mất địa chỉ giao hàng và số điện thoại của khách mà không ai nhận ra cho tới lúc đặt hàng.

Vị trí trong mã nguồn. `backend/server.js:116-118`.

Đề xuất sửa. Xây dựng câu `UPDATE` động, chỉ gồm các trường thực sự có mặt trong body (`Object.prototype.hasOwnProperty.call(req.body, k)`).

API-2 — FR-09 · Áp dụng mã giảm giá

BUG-A2-01 — `POST /api/apply-coupon` không yêu cầu đăng nhập

Mức độ	● Critical
Vi phạm	FR-09 điều kiện C4 — "Đã đăng nhập — Người dùng phải có JWT Token hợp lệ"; SEC-02
Test case	TC-A2-017 , TC-A2-018

Mô tả. Endpoint không gắn middleware `authenticateToken`. Một trong năm điều kiện bắt buộc của FR-09 hoàn toàn không được cài đặt.

Kết quả thực tế.

```
$ curl -X POST localhost:3000/api/apply-coupon -H 'Content-Type: application/json' \
  -d '{"code":"SAVE10","total_amount":500000}'          # KHÔNG kèm Authorization
HTTP/2.0 200
{"success":true,"coupon_id":1,"discount_amount":-450000,"final_amount":500000,...}
```

Tác động. Ngoài việc vi phạm C4, đây còn là bàn đạp cho BUG-A2-04 và BUG-A2-05: vì không có JWT nên hệ thống buộc phải tin vào `user_id` do client gửi, khiến giới hạn số lượt sử dụng trở nên vô nghĩa. Kẻ tấn công cũng có thể dò toàn bộ danh sách mã giảm giá đang hoạt động mà không cần tài khoản.

Vị trí trong mã nguồn. `backend/server.js:360` — `app.post("/api/apply-coupon", (req, res) => {` thiếu `authenticateToken` (so sánh với `/api/coupon-usage` ở dòng 434 thì có).

BUG-A2-02 — Công thức giảm giá percent sai → giảm giá âm

Mức độ	● Critical
Vi phạm	FR-09 — "Loại percent: discount_amount = total × discount_value / 100 "
Test case	TC-A2-032 , TC-A2-033 , TC-A2-036

Mô tả. Mã nguồn tính `total × (1 - discount_value)` thay vì `total × discount_value / 100`. Với `discount_value = 10`, biểu thức thành `total × (1 - 10) = -9 × total`.

Kết quả thực tế.

Đơn hàng	<code>discount_amount</code> kỳ vọng	<code>discount_amount</code> thực tế	<code>final_amount</code> thực tế
500.000 đ	50.000	-4.500.000	5.000.000
1.000.000 đ	100.000	-9.000.000	10.000.000

Tác động. Khách hàng dùng mã giảm giá 10% phải trả **gấp 10 lần** giá gốc. Đây là lỗi tính tiền trực tiếp, ảnh hưởng mọi mã loại percent (SAVE10, EXPIRED). Mã loại fixed (BIGBUY, VIP100) tính đúng — xác nhận qua TC-A2-034, TC-A2-035 đều PASS.

Vị trí trong mã nguồn. `backend/server.js:394-396` và `410-412` (công thức bị lặp lại ở hai nhánh).

```
discount_amount = Math.floor(total_amount * (1 - coupon.discount_value));  
//  
// đúng: Math.floor(total_amount * coupon.discount_value / 100)
```

Đề xuất sửa. Sửa công thức ở cả hai nhánh, và tách hàm tính giảm giá dùng chung để tránh việc phải sửa hai chỗ.

BUG-A2-03 — Ngưỡng đơn hàng dùng > thay vì >=

Mức độ	● High
Vi phạm	FR-09 điều kiện C3 — "Tổng đơn hàng >= (lớn hơn hoặc bằng) min_order_amount "
Test case	TC-A2-013 , TC-A2-015

Kết quả thực tế.

Mã	<code>min_order_amount</code>	Đơn hàng	Kỳ vọng	Thực tế
SAVE10	300.000	299.999	Từ chối	Từ chối ✓
SAVE10	300.000	300.000	Chấp nhận	Từ chối ✗
SAVE10	300.000	300.001	Chấp nhận	Chấp nhận ✓
BIGBUY	500.000	500.000	Chấp nhận	Từ chối ✗

Tác động. Đơn hàng đúng bằng ngưỡng — chính là con số được quảng cáo trên chương trình khuyến mãi ("đơn từ 300.000 đ") — bị từ chối. Khách hàng mua đúng mức yêu cầu vẫn không dùng được mã, gây khiếu nại.

Vị trí trong mã nguồn. backend/server.js:377 — if (total_amount > coupon.min_order_amount) → phải là >= .

BUG-A2-04 — Bỏ user_id là vô hiệu hoá hoàn toàn giới hạn số lượt

Mức độ	● High · Vi phạm FR-09 điều kiện C5 · Test case TC-A2-024
--------	---

Mô tả. Phép kiểm tra số lượt sử dụng nằm trong khối if (user_id) { ... } . Nếu client không gửi user_id , toàn bộ nhánh kiểm tra bị bỏ qua.

Kết quả thực tế (sau khi tài khoản id=2 đã dùng hết 1/1 lượt của SAVE10):

```
có user_id=2    -> {"error": "Bạn đã sử dụng mã này 1 lần (đã đạt giới hạn)"}
Bỏ user_id     -> {"success": true, ...}                ← lách được
```

Tác động. Mã giảm giá "1 lượt/người" trở thành không giới hạn. Kết hợp với BUG-A2-01 (không cần đăng nhập), bất kỳ ai cũng dùng lại mã vô số lần.

Vị trí trong mã nguồn. backend/server.js:384 — if (user_id) { .

Đề xuất sửa. Lấy định danh từ req.user.id (JWT) chứ không từ body; điều kiện C5 phải luôn được kiểm tra.

BUG-A2-05 — IDOR: mượn lượt dùng mã của người khác qua user_id

Mức độ	● High · Vi phạm FR-09 C5 + SEC-02 · Test case TC-A2-025
--------	--

Mô tả. user_id được lấy từ body và không đối chiếu với chủ thể trong JWT. Người dùng đã hết lượt chỉ cần đổi sang user_id của tài khoản khác.

Kết quả thực tế.

```
user_id=2 (đã hết lượt)    -> {"error": "Bạn đã sử dụng mã này 1 lần"}
user_id=1 (tài khoản khác) -> {"success": true, ...}
```

Tác động. Giới hạn số lượt không tồn tại trên thực tế. Vì user_id là số nguyên tuần tự, kẻ tấn công chỉ cần tăng dần để tìm tài khoản còn lượt.

Đề xuất sửa. Như BUG-A2-04 — bỏ hẳn user_id khỏi body.

BUG-A2-06 — Thứ tự kiểm tra điều kiện gây thông báo lỗi sai lệch

Mức độ	Low · Test case TC-A2-010
--------	--

Phép kiểm tra hạn dùng (C2) nằm **bên trong** nhánh kiểm tra ngưỡng (C3). Khi mã vừa hết hạn vừa chưa đủ ngưỡng, hệ thống chỉ báo lỗi ngưỡng.

EXPIRED, đơn 500.000 (≥ ngưỡng) -> "Mã giảm giá đã hết hạn"
EXPIRED, đơn 50.000 (< ngưỡng) -> "Đơn hàng chưa đủ giá trị tối thiểu 100.000"

Tác động. Khách hàng được dẫn đi mua thêm hàng cho đủ ngưỡng, để rồi vẫn không dùng được mã vì mã đã hết hạn từ 2020. Trải nghiệm gây hiểu nhầm, và với bộ phận hỗ trợ thì thông báo này che mất nguyên nhân thật.

Đề xuất sửa. Kiểm tra tuần tự theo đúng thứ tự C1 → C2 → C3 → C4 → C5, mỗi điều kiện một nhánh phẳng.

BUG-A2-07 — Mã giảm giá phân biệt chữ hoa/thường

Mức độ	Low · Test case TC-A2-006
--------	--

SAVE10 → 200 OK , save10 → 404 Not Found .

Tác động. Mã giảm giá được in trên tờ rơi, đọc qua điện thoại hoặc gõ tay. Bắt buộc gõ đúng chữ hoa làm tăng tỉ lệ nhập thất bại, trong khi thông báo trả về ("Mã giảm giá không tồn tại") khiến khách tưởng mã sai.

Đề xuất sửa. So khớp không phân biệt hoa/thường: WHERE UPPER(code) = UPPER(?) .

BUG-A2-08 — Tràn số với total_amount lớn

Mức độ	Medium · Test case TC-A2-044
--------	---

Với total_amount = 1.000.000.000.000.000 , kết quả trả về final_amount = 1000000000000000 — vượt Number.MAX_SAFE_INTEGER (9.007.199.254.740.991). Từ ngưỡng này trở đi mọi phép tính tiền không còn đảm bảo chính xác.

Ngoài ra total_amount hoàn toàn không được kiểm tra: giá trị 0 , số âm, null và chuỗi phi số đều lọt qua (TC-A2-038 ... TC-A2-042 đều FAIL ở nhánh kỳ vọng 400 — xem thêm mục "Ghi chú" cuối tài liệu).

Đề xuất sửa. Kiểm tra Number.isSafeInteger(total_amount) && total_amount > 0 trước khi tính.

API-3 — FR-16 · Import sản phẩm từ CSV

BUG-A3-01 — Người dùng thường import được sản phẩm lên cửa hàng

Mức độ	● Critical
Vi phạm	SEC-03 — "API Admin phải kiểm tra <code>role = 'admin'</code> trong Token, không chỉ kiểm tra sự tồn tại của Token"; FR-12
Endpoint	POST <code>/api/admin/import-products</code>
Test case	TC-A3-040 , TC-A3-041

Mô tả. Endpoint chỉ dùng `authenticateToken` (xác thực) mà không kiểm tra vai trò (phân quyền). Đây đúng là tình huống SEC-03 mô tả.

Kết quả thực tế.

```
# JWT của tài khoản test@eshop.com, role = user
$ curl -X POST localhost:3000/api/admin/import-products -H "Authorization: Bearer $USER_TOKEN" \
  -d '{"products":[{"name":"HANG-GIA-DO-USER-CHEN-23127195","price":1,"category_id":1}]}'
{"message":"Import hoàn tất: 1/1 sản phẩm được thêm","inserted":1,"errors":[]}

# sản phẩm hiện công khai trên cửa hàng:
$ curl "localhost:3000/api/products?search=HANG-GIA-DO-USER-CHEN-23127195"
[{"id":6,"name":"HANG-GIA-DO-USER-CHEN-23127195","price":1,"description":"khong phai admin",...}]
```

Tác động. Bất kỳ khách vãng lai nào đăng ký tài khoản đều chen được hàng lên cửa hàng — **kèm theo tên, giá, mô tả và đường dẫn ảnh do họ kiểm soát**. Kết hợp với BUG-A3-11 (`imageUrl` chấp nhận `javascript:`), đây trở thành đường tấn công stored XSS nhắm vào mọi khách xem trang chủ. Đặt giá **1 đ** còn cho phép thao túng dữ liệu bán hàng.

Vị trí trong mã nguồn. `backend/server.js:188` — `app.post("/api/admin/import-products", authenticateToken, ...)` thiếu bước kiểm tra `req.user.role === 'admin'` . Lưu ý: **toàn bộ** các endpoint `/api/admin/*` khác cũng mắc lỗi này.

Đề xuất sửa. Bổ sung middleware `requireAdmin` và áp cho mọi route `/api/admin/*` .

BUG-A3-02 — Import không nguyên tử — không rollback khi có dòng lỗi

Mức độ	● High
Vi phạm	FR-16 — "Nếu có lỗi ở bất kỳ dòng nào, toàn bộ import phải được rollback (giao dịch nguyên tử — all-or-nothing)"
Test case	TC-A3-032 , TC-A3-033 , TC-A3-034 , TC-A3-036

Mô tả. Mã nguồn duyệt mảng bằng `forEach` và gọi `stmt.run()` cho từng dòng, không mở transaction. Dòng hợp lệ được ghi ngay; dòng lỗi chỉ được đẩy vào mảng `errors` .

Kết quả thực tế.

```
số sản phẩm trước : 6
gửi 1 dòng hợp lệ + 1 dòng THIẾU name:
{"message":"Import hoàn tất: 1/2 sản phẩm được thêm","inserted":1,"errors":["Hàng 3: Thiếu tên sản phẩm"]}
số sản phẩm sau      : 7      ← FR-16 yêu cầu phải vẫn là 6
dòng hợp lệ vẫn nằm trong CSDL:
[{"id":7,"name":"ATOMIC-OK-23127195","price":5000,...}]
```

Kiểm chứng thêm về vị trí dòng lỗi (TC-A3-033 / TC-A3-034): lỗi ở đầu mảng và lỗi ở cuối mảng cho kết quả khác nhau, xác nhận cơ chế là "chạy tuần tự, bỏ qua dòng lỗi" chứ không phải giao dịch.

Tác động. Quản trị viên import file 500 dòng có 1 dòng sai sẽ nhận về trạng thái **nửa vơi**: 499 sản phẩm đã lên sàn, không có cách nào thu hồi ngoài xóa tay. Chạy lại file sau khi sửa dòng lỗi sẽ tạo ra 499 bản ghi **trùng lặp**.

Vị trí trong mã nguồn. backend/server.js:196-228 .

Đề xuất sửa. Bọc trong transaction: db.run("BEGIN TRANSACTION") → validate **toàn bộ** các dòng trước → nếu có bất kỳ lỗi nào thì ROLLBACK , ngược lại COMMIT .

BUG-A3-03 — Không kiểm tra price

Mức độ	● High
Vi phạm	FR-16 — " price phải là số dương"
Test case	TC-A3-018 , -019 , -022 , -023

Mọi giá trị đều được ghi với {"inserted":1,"errors":[]} :

price gửi lên	Vi phạm	Phản hồi
0	không phải số dương	inserted: 1
-50000	số âm	inserted: 1
null	thiếu giá trị	inserted: 1
(thiếu trường)	thiếu trường bắt buộc	inserted: 1

Tác động. Sản phẩm giá 0 đ hoặc **giá âm** lên sàn. Giá âm khiến tổng tiền đơn hàng bị trừ đi — kết hợp với FR-08 (backend nhận total_amount từ client) có thể dẫn tới đơn hàng giá trị âm.

Vị trí trong mã nguồn. backend/server.js:202-205 — chỉ có if (!row.name) , hoàn toàn không kiểm tra price .

BUG-A3-04 — `price` là chuỗi phi số vẫn được ghi vào cột số

Mức độ	● Medium · Test case TC-A3-020
--------	--------------------------------

`price: "khong-phai-so" → {"inserted":1}`. SQLite với kiểu affinity `INTEGER` sẽ lưu nguyên chuỗi khi không ép kiểu được.

Tác động. Đây là kịch bản **mặc định** của tính năng import CSV: mọi trường parse từ CSV đều là chuỗi. Một ô Excel dính chữ ("10.000 đ") sẽ tạo ra bản ghi có giá là chuỗi, khiến mọi phép cộng tổng tiền phía sau nối chuỗi thay vì cộng số.

Đề xuất sửa. `const price = Number(row.price); if (!Number.isFinite(price) || price <= 0) → lỗi;`

BUG-A3-05 — Không kiểm tra khoá ngoại `category_id`

Mức độ	● Medium · Vi phạm FR-15 ("Danh mục: bắt buộc, phải chọn từ danh sách có sẵn") · Test case TC-A3-025 , -026 , -027
--------	--

`category_id` bằng `999`, `0` hoặc `-1` đều được chấp nhận (`inserted: 1`).

Tác động. Sản phẩm gắn vào danh mục không tồn tại sẽ **không hiển thị ở bất kỳ trang danh mục nào** — hàng biến mất khỏi cửa hàng mà không có thông báo lỗi nào cho quản trị viên.

BUG-A3-06 — Tên toàn khoảng trắng lọt qua phép kiểm tra

Mức độ	● Low · Test case TC-A3-013
--------	-----------------------------

Phép kiểm tra là `if (!row.name)`. Chuỗi `" "` là **truthy** trong JavaScript nên lọt qua, tạo ra sản phẩm không tên.

Đề xuất sửa. `if (!row.name || !String(row.name).trim())`.

BUG-A3-07 — Không giới hạn 255 ký tự cho tên sản phẩm

Mức độ	● Low · Vi phạm FR-15 ("Tên sản phẩm: bắt buộc, tối đa 255 ký tự") · Test case TC-A3-015
--------	--

Tên dài 300 ký tự vẫn được ghi (`inserted: 1`).

BUG-A3-08 — Thiếu `category_id` bị âm thầm gán mặc định

Mức độ	● Medium · Test case TC-A3-028
--------	--------------------------------

Dòng thiếu hẳn `category_id` được ghi với `category_id = 1` (`row.category_id || 1` ở `server.js:213`), và báo cáo trả về `{"inserted":1,"errors":[]}` — hoàn toàn không có cảnh báo.

Tác động. Nguy hiểm hơn một lỗi báo sai, vì nó là **lỗi im lặng**: cả lô hàng bị xếp nhầm vào danh mục "Điện thoại" trong khi báo cáo nói import thành công 100%. Không ai phát hiện cho tới khi khách hàng phản ánh.

BUG-A3-09 — Phần tử `null` trong mảng gây crash 500 và lộ stack trace

Mức độ	● High · Test case TC-A3-008
--------	------------------------------

Các bước tái hiện.

```
curl -X POST localhost:3000/api/admin/import-products -H "Authorization: Bearer $ADMIN_TOKEN" \
-d '{"products":[{"name":"OK","price":1000,"category_id":1},null]}'
```

Kết quả thực tế. HTTP 500 kèm trang HTML lỗi của Express:

```
<pre>TypeError: Cannot read properties of null (reading 'name')
    at D:\Kiem_thu\HW6\.sut\eshop-sut\backend\server.js:214:14
    at Array.forEach (<anonymous>);
    at Layer.handleRequest (... \node_modules\router\lib\layer.js:152:17)
```

Tác động. Hai vấn đề: (1) dòng rỗng trong file CSV xuất từ Excel là chuyện thường ngày, và nó làm sập request thay vì báo lỗi tử tế; (2) response **lộ đường dẫn tuyệt đối trên máy chủ, cấu trúc thư mục và phiên bản thư viện** — thông tin hữu ích cho việc tấn công tiếp theo. Lỗi này do assertion cấp collection "*Server không rò rỉ stack trace*" bắt được.

Đề xuất sửa. Kiểm tra `row && typeof row === 'object'` cho từng phần tử; cấu hình error handler chung trả JSON và ẩn stack trace ở môi trường production.

BUG-A3-10 — Mất chính xác với `price` vượt khoảng số nguyên an toàn

Mức độ	● Low · Test case TC-A3-024
--------	-----------------------------

Kiểm chứng. Với giá trị không biểu diễn chính xác được bằng IEEE-754 double:

gửi lên : 9007199254740993

lưu lại : 9007199254740992 (lệch -1)

Tác động. Thấp trong bối cảnh giá hàng thực tế, nhưng là lỗi im lặng: hệ thống không báo bất kỳ cảnh báo nào khi làm tròn. Ghi nhận để hoàn thiện phần kiểm tra miền giá trị.

Lưu ý về cách đọc bằng chứng: giá trị `1e18` dùng trong `reproduce_bugs.sh` *tình cờ* biểu diễn chính xác được bằng `double` nên `round-trip` đúng; phải dùng `9007199254740993` mới thấy được sai lệch. Test case `TC-A3-024` kiểm bằng `Number.isSafeInteger` nên bắt được cả hai trường hợp.

BUG-A3-11 — `imageUrl` chấp nhận giao thức `javascript:`

Mức độ	● Medium · Liên quan SEC-04 · Test case TC-A3-044
--------	---

`imageUrl` gửi lên : `"javascript:alert(document.cookie)"`
`imageUrl` lưu lại : `'javascript:alert(document.cookie)'`

Tác động. Giá trị này được đổ thẳng vào thuộc tính `src` của thẻ `<img>` trên giao diện. Kết hợp với BUG-A3-01 (người dùng thường import được), đây là vector **stored XSS** hoàn chỉnh: kẻ tấn công không cần chạm vào `name` hay `description` — hai trường mà lập trình viên thường nhớ escape.

Đề xuất sửa. Chỉ chấp nhận `imageUrl` có giao thức `http:` hoặc `https:` (hoặc đường dẫn tương đối).

Ghi chú về các test case FAIL không được nâng thành lỗi riêng

Một số test case FAIL cùng chia sẻ một nguyên nhân gốc đã được ghi nhận ở lỗi khác, nên không tách thành mã lỗi mới:

Test case FAIL	Gộp vào	Lý do
TC-A2-038 ... TC-A2-042 (<code>total_amount</code> = 0 / âm / null / chuỗi)	BUG-A2-08	Cùng gốc: thiếu kiểm tra miền giá trị của <code>total_amount</code>
TC-A1-002/003/004/006	BUG-A1-04	Cùng gốc: thiếu kiểm tra <code>name</code>
TC-A1-011 ... TC-A1-020	BUG-A1-03	Cùng gốc: thiếu kiểm tra <code>phone</code>
TC-A3-018/019/022/023	BUG-A3-03	Cùng gốc: thiếu kiểm tra <code>price</code>
TC-A3-025/026/027	BUG-A3-05	Cùng gốc: thiếu kiểm tra khoá ngoại

⚠ Việc sinh viên phải tự làm

Theo §5 và §11 của đề bài, phần dưới đây **bắt buộc phải do người thật thực hiện**, không được tạo tự động:

1. ☒ **Đã tạo GitHub Issue cho từng lỗi** — 24 issue [#5](#) → [#28](#), nhãn `hw06-23127195`. Nội dung gốc: [GITHUB_ISSUES.md](#).
2. **Đính kèm ảnh chụp màn hình** cho mỗi issue: ảnh chụp request/response trong Postman hoặc ảnh chụp terminal khi chạy `reproduce_bugs.sh`. Lưu vào [screenshots/](#).
3. **Ảnh chụp Postman Console** hiển thị dòng log `[X-Student-Id] 23127195 -> ...` — đây là bằng chứng chống gian lận bắt buộc theo §11.